1. The main philosophy I took to the testing of my search routine was that of boundary view analysis. While, to my understanding, it also involved some equivalence partitioning, the partitions were mostly just single-value, rather than testing ranges of correct values. Since I very much was applying the principles of test-driven development for my solution, I was primarily focused on fixing the bugs that arose from the tests I implemented. From there, I completed my implementation of the algorithm and tested the edge cases that I thought I would run into to ensure that my implementation was correct. To accomplish this type of boundary view analysis I employed the use of ZOMBIES, as well as just trying to think of other edge cases that may be relevant. I wanted to check that duplicates, unconventionally sized arrays, even arrays using strings were all covered, which I ultimately think helped me to achieve the goal of my closed box testing.
2. For the handling of unsorted arrays the testing and implementation was mostly open-box, but to ensure maximum thoroughness I was also sure to implement some closed-box tests as well. For the open-box tests, I mostly had to implement a type of check for if values aren't sorted as the algorithm runs, which means that most of the testing was open-box since the amount of inputs I can give to test unsorted arrays are rather limited. For open-box, however, I had to test multiple branches of multiple conditions to ensure that the code for handling unsorted arrays worked properly. Despite this I did still have closed box tests that ensured that unsorted arrays were still handled as described in the instructions.
3. I think that my closed-box tests were vastly more helpful in identifying bugs to fix than my open-box tests. While it's possible that this is skewed because I worked on the closed-box tests first, the fact remains that I still got more utility out of the closed-box testing than the open. From my perspective the reason that the closed box tests were so much more useful is because it was much more streamlined in the process of identifying bugs and fixing them. With closed box tests I simply had to give the program an input

and test it and if it failed then I knew something was wrong and I could go in and fix it. Compared to open box tests, while it allowed me to have more confidence in my breadth of tests it didn't tell me if there were bugs as effectively, although the coverage metric did help me to identify redundancies and areas I needed to refactor. In essence closed-box testing was much more effective at getting a functional program, but open-box testing reinforced the closed-box testing and helped me to write a higher quality implementation.